

Queues in Java

Mostafa Faik

October 6, 2024

Introduction

In this assignment, we explored the implementation of a `queue` data structure using a `linked list` in Java. A `queue` is a fundamental data structure that follows the `First-In-First-Out` (FIFO) principle. This principle is that the first element to be removed from the `queue`, is the first element that was added. This functionality contrasts with a `stack`, which operates on a `Last-In-First-Out` (LIFO) principle.

We began by implementing a basic `queue` that maintains only a head pointer to the first element. We later improved this implementation by introducing a tail pointer for more efficient `enqueue` operations. Throughout the assignment, we conducted tests and benchmarks to evaluate the performance and functionality of both implementations.

Initial Queue Implementation

In the first task, the initial implementation used a singly `linked list`, where each node held a reference to the next node in the `queue`. The `queue` maintained a head pointer, which pointed to the first element in the `queue`, and items were `enqueued` at the end and `dequeued` from the front.

We conducted tests to verify the correctness of the implementation of the code. The tests successfully confirmed that:

- The `queue` returned null when attempting to `dequeue` from an empty `queue`.
- Elements were `enqueued` and `dequeued` in FIFO order, as expected.

Benchmarking the Initial Implementation:

We benchmarked the `enqueue` and `dequeue` operations to assess the performance of the initial `queue` implementation for various `queue` sizes. The following table, shows the result of the benchmarks:

Case	size	Enqueue Time	Dequeue Time
1	100	22.50 ns	19.41 ns
2	200	45.81 ns	20.04 ns
3	400	93.07 ns	21.31 ns
4	800	187.6 ns	22.34 ns
5	1600	372.3 ns	21.27 ns
6	3200	745.1 ns	22.05 ns

Table 1: Initial Queue Benchmark Result

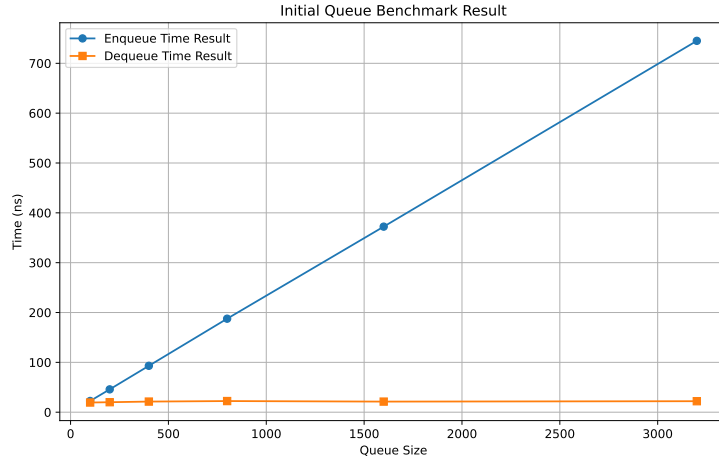


Figure 1: Initial Queue Graph Result

We can observe from the benchmark results and the graph that as the **queue** size increases, the time taken to **enqueue** items grows significantly. This is due to the traversal of the entire **queue** for each **enqueue**. This confirms an $O(n)$ time complexity for **enqueue** operation. On the other hand, the time taken for the **dequeue** operation remains almost constant $O(1)$ because only the head reference is updated, regardless of the **queue** size.

Drawbacks of the Initial Implementation:

Cost of Adding an Element (**Enqueue**):

- In the initial implementation, each **enqueue** operation required traversing the entire list to find the last node. This results in a time complexity of $O(n)$, where n is the number of elements in the **queue**. As the **queue** grows larger, this operation becomes more expensive.

Cost of Removing an Element (Dequeue):

- The **dequeue** operation was efficient with a time complexity of $O(1)$ since it only involved updating the head pointer to point to the next element in the list.

Improved Queue Implementation with Tail Pointer

In this task, we modified the **queue** implementation to include a tail pointer, enhancing the performance and allowing direct access to the last element for enqueue operations. This implementation eliminates the need to traverse the entire list when adding new items. The following table, shows the result of the benchmarks when adding a tail:

Case	size	Enqueue Time	Dequeue Time
1	100	21.26 ns	18.61 ns
2	200	25.04 ns	22.34 ns
3	400	26.27 ns	23.91 ns
4	800	22.56 ns	25.44 ns
5	1600	24.73 ns	25.97 ns
6	3200	26.13 ns	27.09 ns

Table 2: Improved Queue Benchmark Result

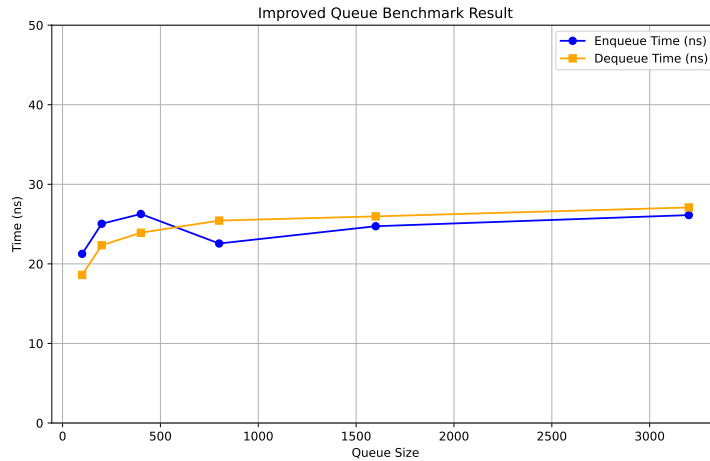


Figure 2: Improved Queue Graph Result

We can observe that with the modified `queue` using both a head and tail pointer, the cost of removing an element (`dequeue`) remains $O(1)$, since removing an item only involves updating the head pointer, regardless of the `queue`'s size. However, the cost of adding an element (`enqueue`) improved from $O(n)$ to $O(1)$ because we can append the new node directly to the tail without traversing the list. So, the new implementation showed significant improved performance in enqueue operations, especially for larger `queue` sizes, while maintaining low dequeue times. The tail pointer effectively optimized the addition of elements without compromising the overall `queue` functionality.

Cost of Operations with Tail Pointer:

Cost of Adding an Element (`Enqueue`):

- With the tail pointer, we can now append new nodes in constant time, making the time complexity for `enqueue` $O(1)$.

Cost of Removing an Element (`Dequeue`):

- The cost of removing an element remains $O(1)$ since the operation only involves updating the head pointer.

Conclusion

In this assignment, we successfully implemented a `queue data structure` using a `linked list` and optimized it by introducing a tail pointer. The initial implementation, had an $O(n)$ time complexity for `enqueue` operations, which made it inefficient as the `queue` size increased. By adding the tail pointer, we reduced the `enqueue` time complexity to $O(1)$, resulting in much faster performance, particularly for larger `queues`.

Through benchmarking, we demonstrated the effectiveness of this optimization. The results highlight the importance of choosing the right data structure to balance performance and functionality in real-world applications. This assignment reinforces the value of understanding and improving the underlying mechanics of data structures to achieve better performance.